SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree.* *(BL4)*

Assessment Tool Weightage: 20%

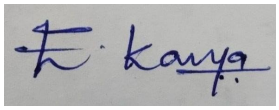
QUESTIONS: **Total Marks:100**

Annexure A

Debugging

Code of Conduct:

I ***E.Kavya(192424365)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Q. No	Understanding of Debugging Concepts (25)	Error Identification(25)	Root Causes Analysis (20)	Error Corrections and Solution Design(20)	Testing and Validation(10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	

	/ 25	/ 25	/ 20	/ 20	/ 10	/ 100

Annexure A

Debugging

Q1 Consider the following syntax-directed definition:

```
E → E1 + T   ( E.val = T.val }

E → T        { E.val = T.val }

T → num      { T.val = num.lexval }
```

For the input:

```
3 + 5
```

the compiler produces:

```
E.val = 5
```

Debug the given syntax-directed definition. Identify the incorrect semantic rule, explain its effect on attribute evaluation, provide the corrected rule, and determine the correct value of the expression.

Q2 . For the expression:

```
a + b * c
```

the compiler constructs the following syntax tree:

```
        *
       / \
      +   c
     / \
    a   b
```

The compiler subsequently generates:

```
t1 = a + b
t2 = t1 * c
```

Debug the syntax-tree construction. Analyze the relationship between operator precedence and the generated tree, construct the corrected syntax tree, and generate the correct intermediate representation.

Q3. For the expression:

```
3 + 4 * 5
```

using:

```
E → E + T | T
T → T * F | F
F → num
```

the compiler annotates the root with:

```
E.val = 35
```

Analyze the annotated parse tree. Trace the synthesized attributes from the leaves to the root, identify the incorrect computation, and determine the correct value of the expression.

Q4 . The symbol table contains:

Identifier	Type
a	int
b	float
c	int
x	float

For the statement:

```
x = a + b * c;
```

the compiler reports:

```
Type Error: incompatible operands
```

Debug the type-checking process. Analyze each subexpression, determine the type of `b * c` and `a + (b * c)`, identify where type conversion is required, and determine whether the assignment to `x` is valid.

Q5. Consider:

```
int a;
float b;
int x;
```

and the statements:

```
x = a + b;
b = a + x;
```

The type checker reports both statements as:

```
Valid
```

and does not generate any conversion.

Analyze the type-checking results. Determine the type of each expression, identify the required conversions, and explain how the type checker should process each assignment.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Debugging Concepts	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Error Identification	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Root Causes Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Error Correction & Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Testing and Validation	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

CO3-AT2 [Debugging]

Name : E.kavya

Reg No : 192424365

Code : CSA1408

Course : Compilers Design

1) Debug the Syntax-Directed Defenition

Given

$E \rightarrow E1+T$ { E.val = T.val }

$E \rightarrow T$ { E.val = T.val }

$T \rightarrow num$ { T.val = num.lexval }

Input 3+5

compiler produces E.val = 5

Error Identification

The incorrect semantic rule is

$E \rightarrow E1+T$ { E.val = T.val }

The rule ignores E1.val since + represents addition, both operands must be included.

Attribute Evaluation

for 3+5:

E1.val = 3

T.val = 5

The incorrect rule gives:

E.val = T.val

= 5

Therefore, the compiler produces the wrong value

correct rule

$E \rightarrow E1+T$ { E.val = E1.val + T.val }

The other rules remain:

$E \rightarrow T \{ E.val = T.val \}$

$T \rightarrow num \{ T.val = num.lexval \}$

Correct Evaluation

$E.val = E1.val + T.val$

$= 3 + 5$

$= 8$

Result: The error was caused by ignoring $E1.val$. The corrected expression value is 8. This matches the source answer's correction.

2) Debug the Syntax Tree and Intermediate Representation

Expression

$a + b * c$

The compiler incorrectly constructs a tree with + below * and generates.

$t_1 = a + b$

$t_2 = t_1 * c$

The assessment specifically asks you to analyze Precedence, correct the tree, and generate the correct intermediate representation.

Error Identification

The * operator has higher precedence than +

Therefore :

a+b*c

must be interpreted as

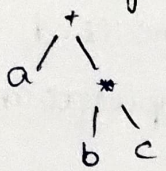
a+(b*c)

The given tree represents

(a+b)*c

So the Syntax tree is incorrect

correct Syntax tree



This represents: a+(b*c)

Error in intermediate code

Given $t_1 = a+b$

$t_2 = t_1+c$

This represents : (a+b)*c

Therefore, it is also incorrect.

correct intermediate Representation

multiplication must be performed first

$t_1 = b*c$

$t_2 = a+t_1$

Result:

The error was caused by incorrect operator Precedence in the Syntax tree and intermediate code. The corrected AST tree and TAC correctly represent a+(b*c).

3) Debug the Annotated Parse tree

Expression

3+4*5

Grammer: E → E+T | F

T → T*F | F

F → num

Compiler gives E.val = 35

The question asks you to trace synthesized attributes, identify the incorrect computation, and find the correct value.

Error Identification

The grammer gives * higher precedence than +.

Therefore : 3+4*5 is evaluated as : 3+(4*5)

Attribute Evaluation

from F → num

we get : F.val = 3

F.val = 4

F.val = 5

For multiplication

T.val = 4*5

= 20

For addition

E.val = 3+20

= 23

Error

The compiler reports → E.val = 35

correct value

4*5 = 20

3+20 = 23

Result: The incorrect annotated value is 35. The correct synthesized attribute value is

E.val = 23

4) Debug the type-checking process

Given

a : int

b : float

c : int

x : float

x = a + b * c ;

The compiler reports: Type Error: Incompatible Operands

Analysis

Since * has higher precedence

a + (b * c)

for b * c : b → float , c → int

The int value c is converted to float.

b * c : float

for a + (b * c):

a → int , b * c : float

Therefore a is converted to float.

a + (b * c) : float

The left side is x : float

Required conversions

c → float

a → float

Correct TAC

t_1 = int to float (c)

t_2 = b * t_1

t_3 = int to float (a)

t_4 = t_3 + t_2

x = t_4

Result: The statement is valid with standard int → float widening conversion. Therefore, the compiler's reported type error is incorrect.

5) Debug the type-checking results

Given int a; , float b; , int x;

x = a+b; , b = a+x;

The compiler reports both statements as valid without generating conversions.

Statement 1 : x = a+b

a : int , b : float

Therefore a+b : float

a must be converted from int to float

But x : int

so assignment requires : float → int

∴ x = a+b; is invalid without explicit conversion

Possible conversion : t_1 = int to float (a)

$t_2 = t_1 + b$

t_3 = float to int (t_2)

$x = t_3$

Statement 2 : b = a+x

a : int ∴ a+x : int

x : int since b : float the required conversion is

int → float

This is a widening conversion and is normally valid.

Possible TAC

$t_1 = a + x$

t_2 = int to float (t_1)

$b = t_2$

final Result : type checker is incorrect in marking both statements as valid without conversions.

Statement	Expression type	Required conversion	Result
x = a+b	float	float → int	invalid
b = a+x	int	int → float	valid